LAPORAN PRATIKUM

STRUKTUR DATA

“STACK“

PEKAN KE-3

disusun Oleh:

Aisyah Najmiatul Fauziah

NIM 2511533030

Dosen Pengampu: DR. WAHYUDI, S.T, M.T

Asisten Pratikum: Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan laporan praktikum Struktur Data mengenai **Stack** ini dengan baik.

Laporan ini disusun sebagai salah satu tugas praktikum untuk memahami konsep dasar struktur data, khususnya struktur data Stack yang menerapkan prinsip **LIFO (Last In First Out)**. Dalam praktikum ini, dilakukan implementasi Stack menggunakan bahasa pemrograman Java sehingga dapat dipahami cara kerja serta penerapannya dalam pengolahan data.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca.

Padang, 10 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR i

DAFTAR ISI ii

BAB I 1

PENDAHULUAN 1

1.1 Latar Belakang 1

1.2 Tujuan Praktikum 1

1.3 Manfaat Praktikum 1

BAB II 2

PEMBAHASAN 2

2.1 Praktikum “NilaiMaksimum_2511533030” 2

2.2 Praktikum “SiswaStack_2511533030” 2

2.3 Praktikum “StackArray_2511533030” 2

2.4 Pratikum “StackArrayDriver_2511533030 3

2.5 Pratikum “StackPostfix_2511533030” 3

BAB III 3

KESIMPULAN 3

3.1 Kesimpulan 3

DAFTAR PUSTAKA 4

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam dunia pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data secara efisien. Salah satu struktur data yang sering digunakan adalah **Stack**, yaitu struktur data yang bekerja berdasarkan prinsip **LIFO (Last In First Out)**, di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

Stack banyak digunakan dalam berbagai aplikasi pemrograman, seperti pada fitur undo dan redo, navigasi halaman pada browser, serta dalam proses evaluasi ekspresi matematika dan algoritma rekursif. Oleh karena itu, pemahaman tentang struktur data Stack sangat penting bagi mahasiswa dalam mempelajari dasar-dasar pemrograman dan algoritma.

Dalam implementasinya, Stack memiliki beberapa operasi dasar seperti **push** (menambahkan data), **pop** (menghapus data), dan **peek** (melihat data teratas). Dengan memahami operasi-operasi tersebut, mahasiswa dapat mengelola data secara lebih terstruktur sesuai dengan prinsip kerja Stack.

Berdasarkan hal tersebut, praktikum ini dilakukan untuk memahami konsep dasar Stack serta mengimplementasikannya dalam bahasa pemrograman Java, sehingga mahasiswa dapat memahami cara kerja dan penerapannya dalam berbagai permasalahan pemrograman.

1.2 Tujuan Pratikum

- Memahami konsep dasar struktur data **Stack**
- Mengetahui prinsip kerja Stack yaitu **LIFO (Last In First Out)**
- Mampu mengimplementasikan Stack dalam bahasa pemrograman Java
- Memahami operasi dasar Stack seperti **push, pop, dan peek**
- Melatih kemampuan dalam mengelola data menggunakan struktur data stack

1.3 Manfaat Pratikum

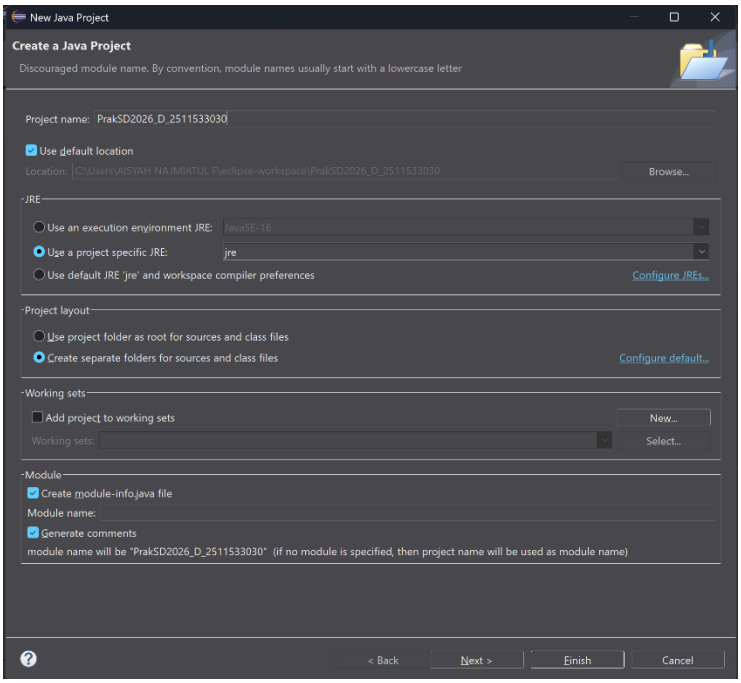
- Mahasiswa dapat memahami cara kerja struktur data Stack
- Meningkatkan kemampuan dalam pemrograman Java
- Mempermudah penyelesaian masalah yang berkaitan dengan konsep LIFO
- Menambah pengalaman dalam implementasi struktur data
- Membantu dalam memahami penerapan Stack pada kasus nyata, seperti undo/redo, navigasi, dan evaluasi ekspresi

BAB II
PEMBAHASAN

2.1 Pratikum “NilaiMaksimum_2511533030”

Langkah Kerja :

1. Buat projek, package dan class baru



2. Buat kode program berikut

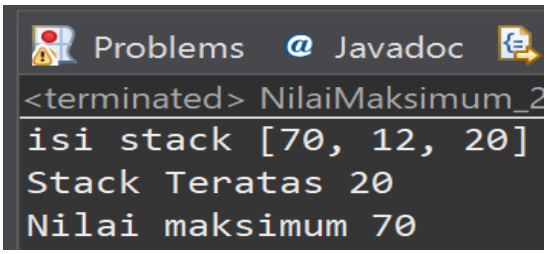
```
package pekan3_2511533030;
import java.util.Stack;
public class NilaiMaksimum_2511533030 {
 public static int max(Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer>();
        int maxValue = s.pop();
        backup.push(maxValue);
        while (!s.isEmpty()) {
               int next = s.pop();
               backup.push(next);
               maxValue = Math.max(maxValue, next);
        }
        while (!backup.isEmpty()) {
               s.push(backup.pop());
        }
        return maxValue;
 }
 public static void main (String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack "+s);
        System.out.println("Stack Teratas "+s.peek());
        System.out.println("Nilai maksimum "+max(s));

 }
```

```
}
```

Gambar 2.1 kode program "NilaiMaksimum_2511533030"

3. Output Program



Gambar 2.1 output program "NilaiMaksimum_2511533030"

Penjelasan Kode

2.1.1 Import Libarary

Digunakan untuk memakai struktur data **Stack** di Java

Stack = struktur data dengan konsep **LIFO (Last In First Out)**

```
import java.util.Stack;
```

2.1.2 Deklarasi class

Mengolah data Stack dan mencari nilai maksimum

```
public class NilaiMaksimum_2511533030 {
```

2.1.2 Method **max()** (inti program)

Mencari nilai terbesar dalam Stack **tanpa menghilangkan isi Stack.**

a) Membuat Stack Cadangan
   - Menyimpan sementara data dari stack utama (s)
   - Supaya data tidak hilang saat di-pop

b) Ambil elemen pertama (TOP)

   pop() → mengambil dan menghapus elemen paling atas

c) Perulangan untuk cek semua elemen

   Proses:

- Ambil semua elemen satu per satu (LIFO)
- Bandingkan untuk cari yang terbesar
- Simpan ke backup

d) Mengembalikan isi stack
   - Karena sebelumnya semua data sudah di-pop (kosong)
   - Maka dikembalikan lagi ke stack awal

e) Mengembalikan nilai maksimum

```
f)  Stack<Integer> backup = new Stack<Integer>();
g)              int maxValue = s.pop();
h)              backup.push(maxValue);
i)              while (!s.isEmpty()) {
j)                      int next = s.pop();
k)                      backup.push(next);
l)                      maxValue = Math.max(maxValue, next);
m)              }
n)              while (!backup.isEmpty()) {
o)                      s.push(backup.pop());
p)              }
q)              return maxValue;
```

```
r)        }
```

2.1.3 Method main() (Program utama)

a) Membuat Stack
   Membuat stack kosong
b) Menambahkan data (Push)
   push() → menambahkan ke atas stack
c) Menampilkan isi stack
d) Menampilkan elemen teratas
   peek(): Melihat elemen paling atas tanpa menghapus
e) Menampilkan nilai maksimum
   Memanggil method **max()**

```
public static void main (String[] args) {
            Stack<Integer> s = new Stack<Integer>();
            s.push(70);
            s.push(12);
            s.push(20);
            System.out.println("isi stack "+s);
            System.out.println("Stack Teratas "+s.peek());
            System.out.println("Nilai maksimum "+max(s));

      }

}
```

2.2 Pratikum "SiswaStack_2511533030"

Langkah Kerja :

1. Buat Class baru yaitu "SiswaStack_2511533030"
2. Buat kode program berikut

```
package pekan3_2511533030;
import java.util.ArrayList;

class Siswa {
    String nama;
    int nim;

    public Siswa (String nama, int nim) {
          this.nama = nama;
          this.nim = nim;
    }

    @Override
    public String toString() {
          return "Nim: " + nim + ", Nama: " + nama;

    }

}
public class SiswaStack_2511533030 {
    private ArrayList<Siswa> stack;

    public SiswaStack_2511533030() {
```

```java
            stack = new ArrayList<>();
    }

    public void push(Siswa mhs) {
            stack.add(mhs);
    }

    public Siswa pop() {
            if (!isEmpty()) {
                    return stack.remove(stack.size() - 1);
            }
            return null;
    }

    public Siswa peek() {
            if (!isEmpty()) {
                    return stack.get(stack.size() - 1);
            }
            return null;
    }

    public boolean isEmpty() {
            return stack.isEmpty();
    }

    public void tampilkanSiswa() {
            for (int i = stack.size() - 1;i >= 0; i--) {
                    System.out.println(stack.get(i));

            }
    }
    public static void main (String[] args) {
            SiswaStack_2511533030 studentStack = new
SiswaStack_2511533030();

            Siswa mhs1 = new Siswa("Ali", 1);
            Siswa mhs2 = new Siswa("Boby", 2);
            Siswa mhs3 = new Siswa("Charles", 3);
            studentStack.push(mhs1);
            studentStack.push(mhs2);
            studentStack.push(mhs3);

            System.out.println("Siswa di dalam stack:");
            studentStack.tampilkanSiswa();

            System.out.println("siswa teratas " +
studentStack.peek());
            System.out.println("mengeluarkan siswa teratas dari
stack: " + studentStack.pop());
            studentStack.tampilkanSiswa();

    }


}
```

Gambar 2.2 kode program "SiswaStack_2511533030"

3. Output Program

```
Problems  @ Javadoc  Declaration  Console ×  Git Staging
<terminated> SiswaStack_2511533030 [Java Application] C:\Users\AISYAH NAJMIATUL
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
siswa teratas Nim: 3, Nama: Charles
mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

Gambar 2.2 output program "SiswaStack_2511533030"

Penjelasan Kode

2.2.1 Class Siswa

Ini adalah **objek/data yang disimpan dalam stack**

nama → nama mahasiswa

nim → nomor mahasiswa

a) Konstruktor

Untuk mengisi data saat objek dibuat

b) Method toString()

Supaya saat di-print hasilnya rapi:

```java
class Siswa {
	String nama;
	int nim;

	public Siswa (String nama, int nim) {
		this.nama = nama;
		this.nim = nim;
	}

	@Override
	public String toString() {
		return "Nim: " + nim + ", Nama: " + nama;

	}

}
```

2.2.2 Class **SiswaStack_2511533030**

a) Atribut

Digunakan sebagai wadah penyimpanan **stack,** Walaupun pakai ArrayList, kita atur supaya berperilaku seperti stack

b) Konstruktur

Inisialisasi stack kosong

```java
public class SiswaStack_2511533030 {
	private ArrayList<Siswa> stack;
```

```
	public SiswaStack_2511533030() {
		stack = new ArrayList<>();
	}
```

2.2.3 Operasi Sack

a) push (Menambah data)
   Menambahkan ke **atas (TOP)**
b) Pop (Menghapus data)
   Menghapus **elemen paling atas**
c) Peek (Melihat TOP)
   Melihat elemen atas tanpa menghapus
d) isEmpty()
   Untuk cek apakah stack kosong
e) Menampilkan Stack
   Ini penting, ditampilkan dari **atas ke bawah**, sesuai konsep stack

```
public void push(Siswa mhs) {
		stack.add(mhs);
	}

	public Siswa pop() {
		if (!isEmpty()) {
			return stack.remove(stack.size() - 1);
		}
		return null;
	}

	public Siswa peek() {
		if (!isEmpty()) {
			return stack.get(stack.size() - 1);
		}
		return null;
	}

	public boolean isEmpty() {
		return stack.isEmpty();
	}

	public void tampilkanSiswa() {
		for (int i = stack.size() - 1;i >= 0; i--) {
			System.out.println(stack.get(i));

		}
	}
```

2.2.4 Method main()(Driver program)

a) Membuat Satck
b) Membuat data siswa
c) Push ke stack
d) Tampilkan semua
e) Peek
f) Pop
g) Tampilkan lagi

```
h) public static void main (String[] args) {
i)            SiswaStack_2511533030 studentStack = new
   SiswaStack_2511533030();
j)
k)            Siswa mhs1 = new Siswa("Ali", 1);
l)            Siswa mhs2 = new Siswa("Boby", 2);
m)            Siswa mhs3 = new Siswa("Charles", 3);
n)            studentStack.push(mhs1);
o)            studentStack.push(mhs2);
p)            studentStack.push(mhs3);
q)
r)            System.out.println("Siswa di dalam stack:");
s)            studentStack.tampilkanSiswa();
t)
u)            System.out.println("siswa teratas " +
   studentStack.peek());
v)            System.out.println("mengeluarkan siswa teratas
   dari stack: " + studentStack.pop());
w)            studentStack.tampilkanSiswa();
x)
y)      }
z)
aa)
bb)}
```

2.3 Pratikum "stackArray_2511533030"

Langkah kerja :

1. Buat class baru "StackArray_2511533030"
2. Buat kode program berikut

```
•  package pekan3_2511533030;
•
•  public class stackArray_2511533030 {
•      static final int MAX_3030 = 1000;
•      int top_3030;
•      int a_3030[] = new int[MAX_3030];
•      boolean isEmpty_3030()
•      {
•            return (top_3030 < 0);
•      }
•      stackArray_2511533030()
•      {
•            top_3030 = -1;
•      }
•      boolean push_3030(int X)
•      {
•            if (top_3030 >= (MAX_3030 - 1)) {
•                  System.out.println("Stack Overflow");
•                  return false;
•            }
•            else {
•                  a_3030[++top_3030] = X;
•                  System.out.println(X + " dimasukkan dalam
   stack");
•                  return true;
```

```
            }
        }
        int pop_3030()
        {
            if (top_3030 < 0) {
                System.out.println("Stack Underflow ");
                return 0;
            }
            else {
                int X = a_3030[top_3030--];
                return X;
            }
        }
        int peek_3030()
        {
            if (top_3030 < 0) {
                System.out.println("Stack Underflow");
                return 0;
            }
            else {
                int X = a_3030[top_3030];
                return X;
            }
        }
        void print() {
            for(int i = top_3030;i>-1;i--) {
                System.out.println(" "+ a_3030[i]);
            }
        }


    }
```

Gambar 2.3 kode program "StackArray_2511533030"

Penjelasan Kode :

2.3.1 Deklarasi Variabel

- MAX_3030 → kapasitas maksimal stack (1000 data)
- top_3030 → penunjuk posisi paling atas (TOP)
- a_3030[] → array untuk menyimpan data stack

```
static final int MAX_3030 = 1000;
        int top_3030;
        int a_3030[] = new int[MAX_3030];
```

2.3.2 Method isEmpty_3030()

fungsi, mengecek apakah stack kosong

```
boolean isEmpty_3030()
        {
            return (top_3030 < 0);
        }
```

2.3.3 Konstruktor

Inisialisasi stack kosong

```
stackArray_2511533030()
	{
		top_3030 = -1;
	}
```

2.3.4 Method **push_3030(int X)**

Menambahkan data ke dalam stack (ke atas), proses nya:
Cek apakah stack penuh, ika iya → **Stack Overflow.** artinya, top naik dulu simpan data ke array.

```
boolean push_3030(int X)
	{
		if (top_3030 >= (MAX_3030 - 1)) {
			System.out.println("Stack Overflow");
			return false;
		}
		else {
			a_3030[++top_3030] = X;
			System.out.println(X + " dimasukkan dalam stack");
			return true;
		}
	}
```

2.3.5 Method **pop_3030()**

Menghapus elemen paling atas

- kalau kosong : Stack Underflow
- kalau tidak kosong : ambilkan data di TOP, turunkan top

```
int pop_3030()
	{
		if (top_3030 < 0) {
			System.out.println("Stack Underflow ");
			return 0;
		}
		else {
			int X = a_3030[top_3030--];
			return X;
		}
	}
```

2.3.6 Method **peek_3030()**

Melihat elemen paling atas tanpa menghapus

```
int peek_3030()
	{
		if (top_3030 < 0) {
			System.out.println("Stack Underflow");
			return 0;
		}
		else {
			int X = a_3030[top_3030];
			return X;
		}
	}
```

2.3.7 Method **print()**

Menampilkan isi stack dari atas ke bawah

```
void print() {
		for(int i = top_3030;i>-1;i--) {
			System.out.println(" "+ a_3030[i]);
```

```
        }
    }
```

2.4 Pratikum "stackArrayDriver_2511533030"

Langkah kerja :

1. Buat class baru "stackArrayDriver_2511533030"
2. Buat kode program berikut

```
package pekan3_2511533030;

public class stackArrayDriver_2511533030 {
    public static void main (String[] args) {
            stackArray_2511533030 s = new stackArray_2511533030();
            s.push_3030(10);
            s.push_3030(20);
            s.push_3030(30);
            System.out.println(s.pop_3030() + " dikeluarkan dari
stack");
            System.out.println("Elemen teratas adalah :" +
s.peek_3030());
            System.out.println("Elemen pada stack :");
            s.print();
    }

}
```

Gambar 2.4 kode program "stackArrayDriver_2511533030"

3. Dapat output sebagai berikut

```
<terminated> stackArrayDriver_25115330
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Elemen pada stack :
 20
 10
```

Gambar 2.4 output program "stackArrayDriver_2511533030"

Penjelasan kode program :

2.4.1 Method **main()**

```
public static void main (String[] args) {
```

2.4.2 Membuat objek Stack

Membuat stack kosong

```
stackArray_2511533030 s = new stackArray_2511533030();
```

2.4.3 Operasi push (menambah data)

Push = menambah ke atas (TOP)

```
s.push_3030(10);
s.push_3030(20);
s.push_3030(30);
```

2.4.4 Operasi pop (menghapus data)

pop = ambil elemen paling atas

```
System.out.println(s.pop_3030() + " dikeluarkan dari stack");
```

2.4.4 Operasi peek (Melihat TOP)

Peek = melihat elemen atas tanpa menghapus

```
System.out.println("Elemen teratas adalah :" +
s.peek_3030());
```

2.4.5 Menampilkan isi stack

Menampilkan dari **atas ke bawah**

```
System.out.println("Elemen pada stack :");
        s.print();
    }

}
```

2.5 Pratikum "StackPostfix_2511533030"

Langkah kerja :

1. Buat class baru "StackPostfix_2511533030"
2. Buat kode program berikut

```
package pekan3_2511533030;
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_2511533030 {
    public static int postfixEvaluate (String expression) {
        Stack<Integer> s_3030 = new Stack<Integer>();
        Scanner input = new Scanner(expression);
        while (input.hasNext()) {
            if (input.hasNextInt()) {  // an operand
(integer)
                s_3030.push(input.nextInt());
            } else {
                String operator = input.next();
                int operand2 = s_3030.pop();
                int operand1 = s_3030.pop();
                if (operator.equals("+")) {
                    s_3030.push(operand1 + operand2);
                } else if (operator.equals("-")) {
                    s_3030.push(operand1 - operand2);
                } else if (operator.equals("*")) {
                    s_3030.push(operand1 * operand2);
                } else {
```

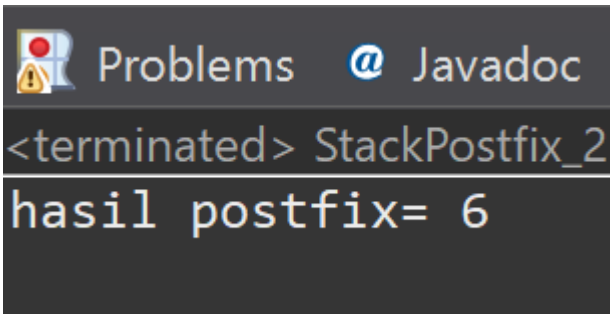
```
                                s_3030.push(operand1 / operand2);
                            }
                        }
                    }
                    input.close();
                    return s_3030.pop();
        }
        public static void main (String[] args) {
                    System.out.println("hasil postfix= "+postfixEvaluate("5
    2 4 * + 7 -"));

        }

    }
```

Gambar 2.5 kode program "StackPostfix_2511533030"

3. Mendapatkan output sebagai berikut



Gambar 2.5 output program "StackPostfix_2511533030"

Penjelasan kode :

2.5.1 Import library

Digunakan untuk:

- **Stack** → menyimpan angka
- **Scanner** → membaca string expression

```
import java.util.Scanner;
import java.util.Stack;
```

2.5.2 Method **postfixEvaluate()**

Menghitung hasil dari ekspresi postfix, cara kerja (konsep stack) :

- Membuat stack kosong
- scanner untuk membaca expression satu per satu

```
Stack<Integer> s_3030 = new Stack<Integer>();
                Scanner input = new Scanner(expression);
                while (input.hasNext()) {
```

2.5.3 Jika angka (operand)

Push ke stack

```
if (input.hasNextInt()) {  // an operand (integer)
                    s_3030.push(input.nextInt());
```

2.5.4 Jika operator

Pop dua elemen terakhir

```
} else {
                    String operator = input.next();
```

```
                        int operand2 = s_3030.pop();
                        int operand1 = s_3030.pop();
```

2.5.5 Proses operasi

Setelah dihitung, hasil dimasukkan lagi ke stack (**push**)

```
s_3030.push(operand1 + operand2);
                        } else if (operator.equals("-")) {
                              s_3030.push(operand1 - operand2);
                        } else if (operator.equals("*")) {
                              s_3030.push(operand1 * operand2);
                        } else {
                              s_3030.push(operand1 / operand2);
                        }
                  }
            }
            input.close();
```

2.5.6 Hasil akhir

```
return s_3030.pop();
      }
```

2.5.7 Method main()

Menjalankan program dengan input postfix

```
ublic static void main (String[] args) {
            System.out.println("hasil postfix= "+postfixEvaluate("5 2 4 * + 7 -
"));

      }

}
```

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Stack merupakan struktur data yang bekerja dengan prinsip LIFO (Last In First Out), yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan.

Pada program yang telah dibuat, Stack diimplementasikan dalam beberapa bentuk, yaitu menggunakan array, ArrayList, dan Stack bawaan Java. Setiap implementasi menunjukkan operasi dasar Stack seperti push, pop, peek, dan isEmpty yang digunakan untuk mengelola data secara terstruktur.

Selain itu, Stack juga diterapkan dalam kasus nyata seperti perhitungan ekspresi postfix, di mana Stack digunakan untuk menyimpan operand sementara dan melakukan operasi berdasarkan urutan tertentu. Hal ini menunjukkan bahwa Stack tidak hanya digunakan untuk penyimpanan data, tetapi juga sangat berguna dalam penyelesaian masalah algoritma.

Dengan demikian, pemahaman mengenai Stack sangat penting karena dapat membantu dalam pengolahan data secara efisien serta menjadi dasar dalam mempelajari struktur data dan algoritma yang lebih kompleks.

DAFTAR PUSTAKA

Wahyudi, Dr., S.T., M.T. (2026). *Materi Struktur Data: Stack*. Bahan ajar PowerPoint.
Oracle. (2023). *Java Platform Standard Edition Documentation*. Diakses dari: https://docs.oracle.com/javase/